

An Efficient FPGA-Based Edge AI System for Railway Fault Detection

Yulin Fu, Dongwei Yan, Jiale Li, Sean
Longyu Ma, and Chiu-Wing Sham
The University of Auckland

Hong-fu Chou
University of Luxembourg

Abstract—Binary neural networks (BNNs), characterized by 1 bit-width weights and activations, are well-suited for deployment on edge computing platforms such as field-programmable gate arrays (FPGAs), especially for real-time monitoring and detection tasks due to their efficient bitwise arithmetic and minimal memory requirements. This makes them ideal for energy-sensitive consumer electronics, where edge computing lowers latency, enhances privacy, and conserves bandwidth by enabling real-time decision-making and localized data processing. In this study, we leverage the strengths of BNNs to develop a real-time railway fault detection system on FPGAs. To address the imbalanced resource utilization that limits model implementation in resource-constrained scenarios using FPGAs, we have developed a digital signal processor (DSP) packing algorithm that optimizes DSP usage and reduces reliance on lookup table resources. Our experiments demonstrate that, with optimized FPGA resource utilization, the system achieves state-of-the-art accuracies of 90.9% and 93.4%, respectively. Furthermore, our FPGA-based system outperforms central processing units and graphics processing units, achieving $101.9\times$ and $33.8\times$ improvements in cost-energy efficiency. This advancement in FPGA-based system showcases the potential of edge computing in transforming consumer devices by enabling sophisticated functionalities while maintaining high energy efficiency. We release our source code to encourage further exploration in this direction, and the code is available at: https://github.com/JiuYing2235/BNN_Railway.git.

Digital Object Identifier 10.1109/MCE.2025.3527118

Date of publication 8 January 2025; date of current
version 16 June 2025.

■ **EDGE COMPUTING REFERS** to a distributed computing paradigm that brings data processing closer to its source.¹ As the Internet of Things (IoT) continues to grow rapidly, connected devices and sensors around the world are generating massive amounts of data, leading to increasing demands for efficient and real-time data processing and analysis.^{2,3} Edge computing addresses these challenges by enabling real-time data collection and processing capabilities, even in remote or high-risk environments. In this context, platforms such as field-programmable gate arrays (FPGAs) have proven to be highly effective, offering low latency, efficient bandwidth utilization, and customizable computational features.⁴ These characteristics make edge computing particularly advantageous in consumer electronics, where space efficiency, cost-effectiveness, and low power consumption are critical.⁵ Through hardware-software co-design, manufacturers can develop consumer devices that achieve real-time performance while maintaining energy efficiency and high computational capabilities. One important application of edge computing is railway fault detection, where real-time processing significantly enhances the detection efficiency and effectiveness of railway defects.

Railway accidents can cause severe damage to the public, making the assurance of safe railway operations a critical issue. Traditionally, fault detection methods were performed manually, which was both time-consuming and costly. Recently, deep convolutional neural networks (DCNNs) have emerged as a promising solution for the detection of railway defects. Tian et al.⁶ developed an FPGA-based track fastener detection system using YOLOv2. The improved fastener detection system demonstrated advantages in terms of processing time and hardware resource utilization compared to embedded maintenance systems. However, the detection time of 41 ms does not satisfy the real-time processing requirements for 60 fps video streams. Another FPGA-based railway fault detection system⁷ was proposed, where the researchers employed a traditional CNN model for fault detection and leveraged FPGA acceleration. Nonetheless, this approach relied on a more expensive

FPGA with higher resource capacity while achieving an accuracy of only 88.9%. To address these limitations, this study focuses on the deployment of a binary neural network (BNN) on an FPGA platform. BNNs, characterized by 1 bit-width weights and activations, allow for efficient implementation with low power consumption, making them ideal for edge devices. We also introduce a digital signal processor (DSP) packing algorithm that significantly optimizes the use of FPGA resources, balancing the usage of DSP and lookup table (LUT) resources. This enables real-time railway fault detection with improved resource efficiency, making our system more suitable for edge computing environments compared to conventional methods. The following are the major contributions of this work:

- 1) A novel DSP packing algorithm is proposed, which reduces LUT resource utilization by 10% while DSP usage only increased by 4 and 7 slices, thus significantly optimizing hardware resource utilization. Compared to traditional BNN model deployment, this packing strategy addresses a major challenge in deploying models on resource-constrained FPGAs for consumer-based applications, enabling effective balancing of resource efficiency and parallelism.
- 2) Two BNN-based models for railway fault detection (BRPnet2 and RBPnet4) have been designed to classify and detect faults on the railway, which achieve state-of-the-art (SOTA) detection accuracies of 90.9% and 93.4%, respectively. The DSP packing algorithm introduced in this work is applied within these models, significantly improving resource efficiency without affecting the computational complexity of the models.
- 3) We present an efficient railway fault detection system that utilizes a hardware-software co-design approach, specifically designed for edge computing platforms. The cost-energy efficiency of our FPGA-based approach outperforms traditional CPU and GPU solutions by approximately 101.9× and 33.8×, respectively. This system offers a more cost-energy efficient application prospect for consumer electronics compared to conventional high-cost computing devices.

RELATED WORK

In recent years, various methods have been developed for detecting railway defects, including ultrasonic detection,⁸ laser ultrasonics, and eddy current detection. Jiang et al.⁹ proposed a rail surface crack detection method based on laser ultrasonic technology combined with variational mode decomposition (VMD). The use of VMD improves the signal-to-noise ratio (SNR) by decomposing ultrasonic signals into multiple intrinsic mode functions, allowing a clearer visualization of the defect-induced wave energy. In addition, nonneural network detection techniques such as ultrasonic detection and eddy current detection are also widely used in railway fault detection. These technologies are hardware-centric, leading to high costs and complex deployment processes. An alternative approach is machine vision detection. Chen et al.¹⁰ introduced an innovative vision-based method for detecting fastener defects, using a cascaded approach with three detection stages based on DCNNs, progressing from coarse to fine analysis. In addition, a railway defect detection system based on track geometry data has been proposed, utilizing both supervised and unsupervised machine learning techniques.¹¹ Wu et al.¹² proposed a hybrid deep learning architecture for rail surface segmentation and surface defect detection, which integrates fully convolutional networks (FCN) and image processing techniques. The use of a hybrid loss function that combines binary cross-entropy, structural similarity index measure (SSIM), and intersection-over-union (IoU) further improved the detection precision of rail surface defects. These studies on railway fault detection have relied primarily on neural network models for feature extraction and fault detection. However, deploying these methods on edge computing platforms remains challenging due to their high computational complexity and large number of parameters. BNNs, with 1-bit weights and activations, present a promising solution for deep learning inference on resource-constrained edge devices. Equation (1) provides the details

$$W_i, A_i = \begin{cases} -1, & \text{if the value is negative} \\ +1, & \text{if the value is positive} \end{cases} \quad (1)$$

where W_i represents the weights and A_i represents the activations. In recent years, numerous studies have explored the FPGA acceleration of BNNs. Notably, FracBNN uses fractional activations, achieving a top-1 accuracy of 71.8% in ImageNet classification.¹³ HyBNN employs a hybrid BNN (FuseBNN) and 4-Bit-Net design to mitigate accuracy loss.¹⁴ The authors of ReBNet¹⁵ introduced residual binarization as a reconfigurable dimension of BNN, along with an activation function with several scaling factors, allowing ReBNet¹⁵ to maintain the simplicity of the hardware architecture while achieving higher accuracy. However, these methods have been studied primarily in a theoretical context and have not been implemented in practical applications such as railway fault detection, with experiments typically conducted on resource-rich FPGAs. In this study, we address the challenge of deploying BNN models on small, resource-constrained FPGAs. To achieve this, we developed a BNN-based railway fault detection system, incorporating techniques such as pruning and quantization to compress the model.

METHODOLOGY

In this section, we present the overall system, the models we developed, and the hardware operations in sequence.

Overall Architecture

One of the key contributions in our work is the complete system architecture introduced in Figure 1. This system includes three main steps: image acquisition, image processing, and host computer communication. Figure 1(a) provides a system-level overview, and the FPGA (Zynq 7Z020) is established as the core component responsible for accelerating computational tasks. The specific process of image acquisition includes utilizing the OpenCV library to perform the task. Meanwhile, the captured images are encoded into binary files and stored in off-chip memory, and the PYNQ environment is utilized to manage OpenCV, communication, data transfer, and processing on the FPGA. Figure 1(b) illustrates the overall architecture of the accelerator. As in the programmable logic section of FPGA, the entire cycle of an accelerator

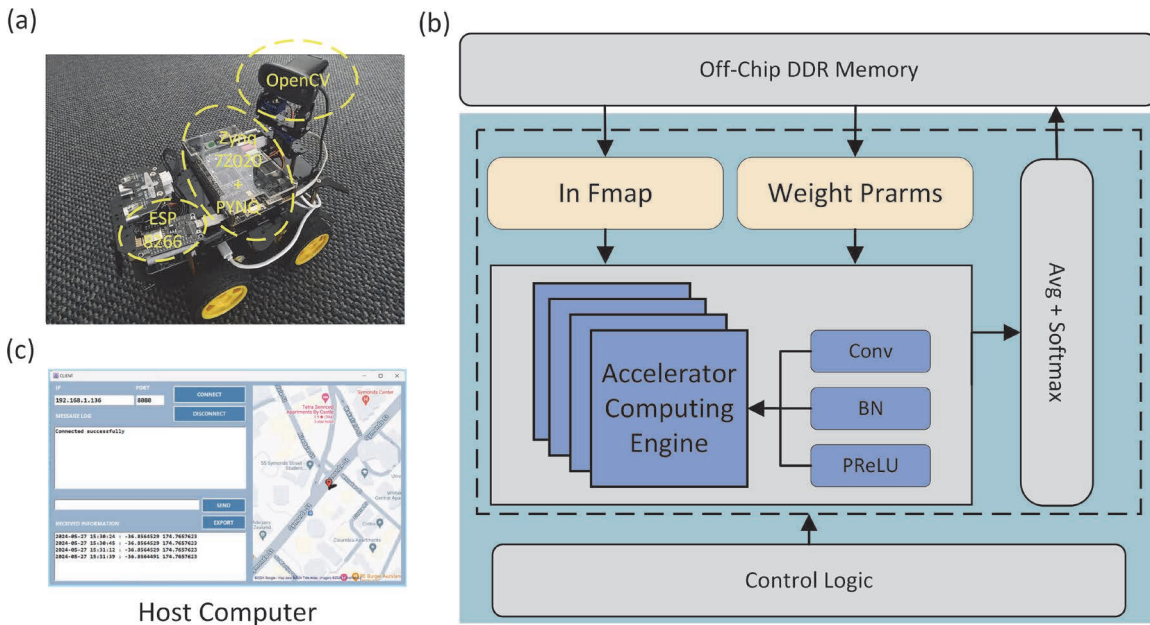


FIGURE 1. Railway faults detection system. (a) System components. (b) Hardware accelerator computation module. (c) Communication module and host computer interface.

computing engine starts with fetching the binary encoded feature maps and weights from the off-chip DDR memory. In order to balance between parallelism and resource utilization, we select pipeline and unroll directives to enhance the execution of the BNN network. Meanwhile, auxiliary functions, including avgpool, fully connected, and softmax, are prepared to classify the state of the faults. After that, we manage to store the result of classification on the processing system for further processing. The FPGA-based system can be integrated with existing railway monitoring equipment, such as sensors and communication modules. Upon detecting a fault in the railway images, the TCP/IP protocol is utilized to transmit the geographical coordinates and timestamp of the fault to the host computer for engineers to repair the faults. Figure 1(c) outlines the host computer interface. This approach leverages the reconfigurability of FPGAs to adapt the system to various railway environments. Detection algorithms can be customized for specific track geometries or environmental challenges by updating the FPGA bitstream, allowing the system to handle diverse operational scenarios without requiring hardware modifications. This reconfigurability ensures that the system remains flexible and scalable, providing real-

time monitoring and rapid response to detected faults in a wide range of railway environments.

Model Design

For our case study, we use the railway dataset constructed by Eunus et al.¹⁶ The original dataset has 428 images in total, containing 156 defective and 272 non-defective images. These images capture three main types of railway faults: rail breaks, missing components, and absent fasteners. The dataset includes images under various conditions, such as daytime, nighttime, and overcast weather. These diverse scenarios facilitate a comprehensive evaluation of the robustness of the railway fault detection model against variations in lighting conditions in real-world environments. The images in the dataset are captured from different camera angles, ensuring compatibility with most camera sampling positions. To make our experiment and the proposed method more reliable, various types of image augmentation were applied, including blurring, noise addition, random flipping, and rotation. Thus, the final training set was expanded to 49 761 images, as shown in Figure 2, which illustrates the types of railway dataset. The adoption of these data augmentation techniques led to an accuracy improvement of 3%



FIGURE 2. Samples of railway defect images, including missing components, rail breaks, and absent fasteners. (a) Image size: 224×224 . (b) Image size: 32×32 .

during the training process, as the model was able to learn more diverse features by simulating various real-world railway scenarios.

We propose two new ResNet-based BNN models (RBPnet2 and RBPnet4) for railway fault detection and compare our results with conventional models to demonstrate their potential in image classification. The basic block used in our BNN models is illustrated in Figure 3, which consists of BinaryConv layers (BC), BatchNorm (BN) layers, PReLU layers, and Avgpool layers. Different from FracBNN,¹³ we use PReLUs instead of BPreLUs, due to BPreLU resulting in greater

storage costs with extra X-bias and Y-bias, which are redundant for rail fault detection. On the 32×32 dataset, we designed the RBPnet2 BNN model with two of our proposed basic blocks. These blocks are followed by average pooling, fully connected layers, and softmax to output classification probabilities. To complete the feature extraction task, a 3×3 binary convolution was applied in RBPnet2, while a 1×1 kernel was used in RBPnet4. As a result, most BNN networks work on bitwise arithmetic, which leads to significant LUT resource overhead. The quantity of resources is typically fixed on FPGAs, and excessive use of any single resource will impede model deployment. To address this challenge, we implemented a DSP-based packing operation within the BN layer, and the details are provided in the following sections. In the 224×224 dataset, we designed the RBPnet4 BNN model with four of our proposed basic blocks, and it has an additional three fully connected layers to improve the classification accuracy.

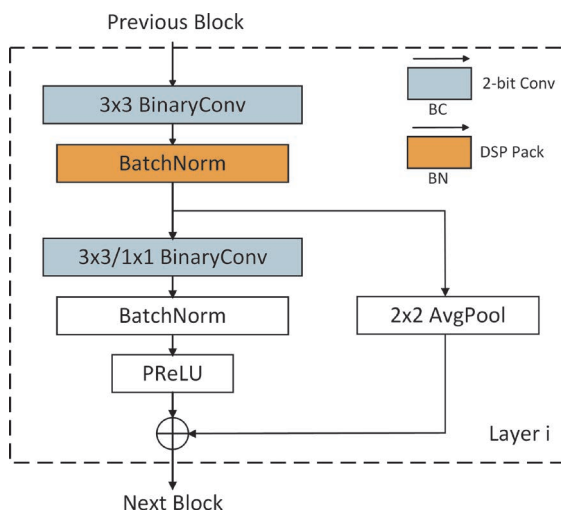


FIGURE 3. Basic binary block in RBPnet2 and RBPnet4.

BNN Convolution Operations

The specific execution flow of the binary convolution is shown in Figure 4. Different from weights and activations, we store the feature map in on-chip Block RAM. Each convolutional layer consists of the most significant bit (MSB) phase and the least significant bit (LSB) phase. During MSB and LSB computations, the 3×3 kernel slides over the feature maps. Furthermore, it

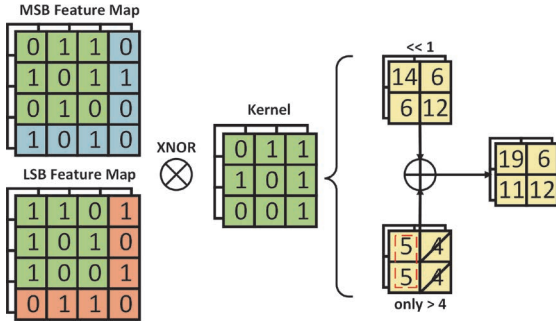


FIGURE 4. Binary convolution computation procedure.

trades massive multiply accumulate operation (MAC) for logical XNOR and pop-count operations to produce the preliminary convolution output. As indicated in the following:¹⁷

$$O_{ij} = P\left(\sum_{k=1}^n \text{XNOR}(\text{Bin}(X_{ijk}, W_k))\right) \quad (2)$$

where O_{ij} is the output at position (i, j) ; $P(\cdot)$ is the pop-count function that counts the number of ones; and n is the number of input channels. For the outputs of the LSB feature map, a threshold Δ is established. If the LSB output exceeds this threshold, it indicates that the value is considered to contribute more to the model accuracy, as demonstrated in the following:¹³

$$O = \begin{cases} O_{msb} \ll 1 & O_{lsb} \leq \Delta \\ (O_{msb} \ll 1) + O_{lsb} & O_{lsb} > \Delta \end{cases} \quad (3)$$

This structure ensures that even the weights and feature map data are quantized to low bit-widths, the model retains high accuracy.

DSP Packing

FPGAs consist of programmable fabric such as LUTs, Flip-Flops (FFs), and non-programmable parts like DSPs which is an essential resource for performing MAC operations.¹⁸ Research indicates that optimizing the usage of DSP resources for computationally intensive tasks is crucial to improving performance, minimizing power consumption, and conserving chip area.¹⁹ During the computation of BNN, a significant amount of bitwise arithmetic is required, which results in leaving most of the DSP's resources unused. The traditional INT4 packaging method essentially calculates the outer product of two vectors, activations a and weight w , as

described in the following Equation:¹⁹

$$a \cdot w^T = \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} \cdot \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}^T \quad (4)$$

Unfortunately, our attempt to apply INT4-packing in the BN layer resulted in a 46.1% reduction in precision within our model, due to information loss in the binary convolutional layers of BNNs, which depend on BN functions to correct feature information. Thus, we designed a fixed 12 bits (FIXED12) packing to generalize INT4-packaging, which is specifically tailored for BN layers.

Our DSP packing method aims to combine multiple MAC operations into DSP units to fully utilize hardware resources and improve computational efficiency. In the process shown in Figure 5, we pack two feature map values (α_0 and α_1) and their corresponding two weight values (w_0 and w_1) into higher bit-width data (α and w), thereby achieving parallel computation. Specifically, we first perform bit-shifting operations and add the values to place each value at a specific bit position, resulting in the packed values:

$$\alpha = \alpha_0 \cdot 2^{B_{\alpha_0}} + \alpha_1 \quad w = w_0 \cdot 2^{B_{w_0}} + w_1 \quad (5)$$

Here, B_{α_0} and B_{w_0} represent the bit offset positions for the value of the feature map α_0 and the weight value w_0 , respectively. Typically, B_{α_0} and B_{w_0} are set to 24, which means that α_0 and w_0 are shifted to occupy the higher 12 bits of the packed 32-bit data, while α_1 and w_1 remain in the lower bit positions. In this way, α_0 and α_1 (as well as w_0 and w_1) are effectively “stacked” together. After packing, we input the packed values α and w into the DSP for multiplication, thus utilizing the DSP's high-efficiency multiplication capabilities to perform parallel computation. The result of the multiplication can be expressed as:

$$\begin{aligned} Result &= (\alpha_0 \cdot 2^{B_{\alpha_0}} + \alpha_1) \times (w_0 \cdot 2^{B_{w_0}} + w_1) \\ &= \alpha_0 w_0 \cdot 2^{B_{\alpha_0} + B_{w_0}} \\ &\quad + (\alpha_0 w_1 + \alpha_1 w_0) \cdot 2^{B_{\alpha_0}} + \alpha_1 w_1 \end{aligned} \quad (6)$$

In the result, $\alpha_0 w_0$ occupies the higher part of the result (at position $B_{\alpha_0} + B_{w_0}$), while $\alpha_1 w_1$ with offset 0. Thus, a single packed multiplication effectively completes two independent multiplication operations, $\alpha_0 \times w_0$ and $\alpha_1 \times w_1$. After the multiplication, we can unpack the result to

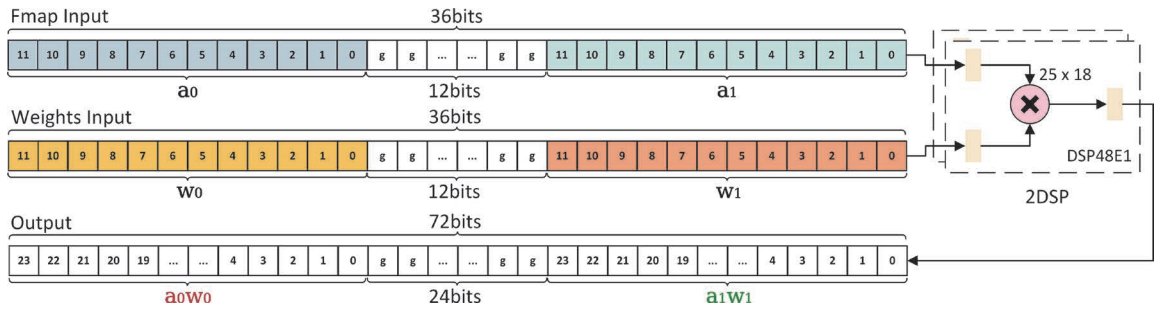


FIGURE 5. FIXED12-packing for performing two parallel multiplications on two DSPs.

extract each individual product. For the high-bit product $\alpha_0 \times w_0$, we extract the higher 24 bits of the result, while the low-bit product $\alpha_1 \times w_1$ extracts from $[24, 0]$.

In addition, we use unsigned types during computations, due to the results of the packed values requiring deviation correction when performing computations with signed types, and a mask bit sa/sw has been applied to preserve the sign of the input data. The Algorithm 1 outlines the details. Since the input data are in a fixed point format with 16 fractional bits, the final result is shifted to the right by 16 bits to restore the correct value. This approach provides a reliable and efficient method for packing DSPs in BNN models, balancing the resources on FPGAs, so that we can fully utilize the resources of a resource-constrained FPGA.

EVALUATION

In this section, we first outline the model configuration and precision outcomes, and then present our results on the FPGAs.

Algorithm 1. DSP Packing Multiplication Operation.

- 1: **Input:** $fmap, bn_weight$, Channel ch
- 2: **Output:** $result_0, result_1$
- 3: $sa_0 \leftarrow \text{sign}(fmap[ch])$
- 4: $sa_1 \leftarrow \text{sign}(fmap[ch + 1])$
- 5: $sw_0 \leftarrow \text{sign}(bn_weight[ch])$
- 6: $sw_1 \leftarrow \text{sign}(bn_weight[ch + 1])$
- 7: $pack_a \leftarrow ((a_0 \wedge 0xFFF) \ll 24) | (a_1 \wedge 0xFFF)$
- 8: $pack_w \leftarrow ((w_0 \wedge 0xFFF) \ll 24) | (w_1 \wedge 0xFFF)$
- 9: $result_pack \leftarrow pack_a \times pack_w$
- 10: $result_0 \leftarrow result_pack[71 : 48] \gg 16$
- 11: $result_1 \leftarrow result_pack[23 : 0] \gg 16$
- 12: Apply sign correction to $result_0, result_1$

Model Setup and Results Comparison

We evaluated RBPnet2 and RBPnet4 in 32×32 and 224×224 railway fault detection datasets, respectively. RBPnet2 features approximately 27.132 million total MACs and 65.762 thousand parameters, while RBPnet4 has around 629.408 million total MACs and 34.558 million parameters. We augment the input images using resizing and random flipping. For both dataset sizes, we train the model for 400 epochs in each training step, and the batch size is 64. The initial learning rate for the 32×32 dataset is $1e-3$ in each epoch, while for the 224×224 dataset, the learning rate is $5e-4$. All training experiments are completed on NVIDIA RTX 4080 GPUs. To optimize the model structure, we introduced an entropy-based pruning technique which dynamically updates a threshold during training to evaluate each layer's contribution to the network,²⁰ resulting in an effective reduction in the depth of the model. The quantized BNN model compared to the full-precision model, achieves a substantial reduction in model size with only a 0.3% accuracy loss, making it suitable for deployment on resource-constrained platforms. As demonstrated in Table 1, on the same datasets and image sizes, the F1-Score of our RBPnet4 model increased by 0.01 to 0.07, achieving the highest precision for detecting defective items across the entire dataset with an impressive F1-Score of 0.94 for positive class and 0.93 for negative class. These results indicate that RBPnet4 outperformed all other models in evaluation.

FPGA-Based Hardware Design

We evaluated the performance of the RBPnet2 and RBPnet4 models on both the Zynq 7Z020 and Xilinx KV260 development boards. The Zynq 7Z020 offers 53.2K LUTs, 220 DSPs, and

TABLE 1. Accuracy comparison of different models.

Metrics	InceptionResNetV2 ¹⁶	InceptionV3 ¹⁶	Xception ¹⁶	ECARRNet ¹⁶	RBPnet4
Precision (D)	0.88	0.89	0.93	0.95	0.98
Precision (N-D)	0.84	0.91	0.90	0.89	0.98
Recall (D)	0.84	0.90	0.90	0.89	0.90
Recall (N-D)	0.88	0.89	0.93	0.95	0.89
F1-Score (D)	0.86	0.90	0.91	0.92	0.94
F1-Score (N-D)	0.86	0.90	0.91	0.92	0.93
Overall F1-Score	0.86	0.90	0.91	0.92	0.93

* D is Detection, and N-D is Non-Detection.

4.48Mb of BRAM, while the KV260 provides 128K LUTs, 1336 DSPs, and 11.6Mb of BRAM. We implemented hardware accelerators for RBPnet2 and RBPnet4, running the models fully unrolled on the boards. RBPnet2 operated at a clock frequency of 100 MHz, while RBPnet4 ran at 250 MHz. Table 3 summarizes the resource utilization of both models in each development board. The FPGA-based platform is integrated into a line-following vehicle equipped with supplementary lighting to facilitate image acquisition at night. The onboard camera employs a hand-eye calibration approach²¹ to lock onto the central position of the railway track from a lateral perspective.

Figure 6 highlights the impact of the DSP packing algorithm on FPGA resources for a single BN layer. We achieved approximately a 10% reduction in LUTs resource utilization, while DSP usage only increased by 4 and 7 slices, respectively. This method enabled us to deploy models on resource-constrained FPGAs while resource utilization is imbalanced. Table 2 compares the performance of our model accelerators with CPUs and GPUs under the same conditions. Here we present (7) to calculate the performance per kilowatt per dollar.

$$CEES = \frac{T_{out}}{P_{total} \times C_{total}} \quad (7)$$

where *CEES* represents the cost-energy efficiency score, T_{out} is the system throughput, P_{total} denotes the total power consumption, and C_{total} represents the total cost. In the field of consumer electronics, it is commonly understood that higher throughput is preferable,

while power consumption and cost should be minimized as much as possible. Thus, we can assert that a higher *CEES* indicates greater performance. Notably, our design demonstrated significant advantages in terms of power consumption and energy efficiency. As an edge computing platform, it also offers strong cost competitiveness. In terms of cost-energy efficiency, the Zynq 7Z020 demonstrates approximately a 56.2× improvement over the CPU and a 59.6× improvement over the GPU. The KV260 also exhibits significant performance enhancements, achieving about 101.9× greater efficiency compared to the CPU and approximately 33.8× greater efficiency than the GPU. Our system achieves a 4× recognition speed increase compared to the HW-based approach⁶ and 22.4× energy efficiency in comparison to previous work.⁷ After mapping the models to the FPGA, the resource utilization is detailed in Table 3. LUTs were used primarily for convolution operations, while DSPs handled normalization and activation.

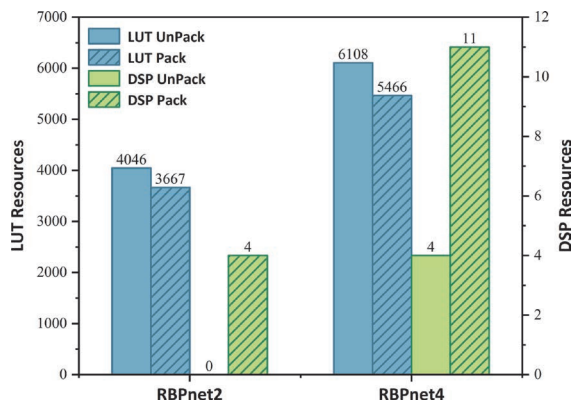


FIGURE 6. Resources changes within a single-layer BN.

TABLE 2. Comparison of FPGA, CPU, and GPU.

Platform	FPGA		CPU		GPU	
	Zynq 7Z020(Ours)	KV260(Ours)	i7 14700KF		RTX 4080	
Price (US dollar)	172.88	422.46	521.16		1482.00	
Release Date (Year)	2010	2021	2023		2022	
Technology (nm)	28	16	10		5	
Frequency (MHz)	100	250	5500		2205	
Dataset Size (Pixel)	32× 32	224× 224	32× 32	224× 224	32× 32	224× 224
Latency (ms)	0.7	10.0	0.5	38.5	0.2	1.9
Throughput (GOPs)	77.52	125.88	108.53	32.69	258.41	662.53
Power (W)	2.3	3.9	60.1	83.4	53.4	197.5
Energy Efficiency (GOPs/W)	33.70	32.28	1.81	0.39	4.84	3.35
CEES (GOPs/kW/Dollar)	194.93	76.41	3.47	0.75	3.27	2.26

* The prices are taken from Octopart as of September 24, 2024.

The weight and feature map data are stored in off-chip memory so that we can deploy the model on FPGAs more economically, since off-chip memory is cheap. For a 60 frames-per-second (fps) video stream, each frame must be processed within approximately 16.7 ms to maintain real-time performance. For more constrained consumer electronics devices such as the ARM with four-core A73, it can also complete the detection task. However, it will cost 10 s detection time,²² which cannot satisfy the requirements of real-time processing. Our architecture achieves model inference times of 0.7 ms for RBPnet2 and 10 ms for RBPnet4, respectively. Even when factoring in an additional time of 1 to 5 ms for image transmission,²³ both designs are capable of accelerating detection sufficiently to meet the real-time requirements of 60 fps video streams, ensuring that every frame is processed well within the necessary time constraints for practical applications.

TABLE 3. Resource utilization after pack.

Resource	RBPnet2	RBPnet4
DSPs	104 (47.3%)	160 (12.8%)
BRAM	86.5 (61.9%)	144 (100%)
LUTs	41491 (77.9%)	65492 (55.9%)

LIMITATIONS AND FUTURE WORK

Despite the significant advancements in FPGA-based railway fault detection, the proposed system still faces certain limitations that require further exploration. To ensure compatibility with the dataset, all images collected are preprocessed at a resolution of 224×224, which facilitates model training and testing. Furthermore, to validate the scalability of our approach, we have customized and tested models using smaller image sizes, such as 32×32. However, extreme environmental variations can affect the detection accuracy of the system. FPGA-based preprocessing is not always capable of fully compensating for such variations, potentially leading to performance degradation. Future research will focus on actual commissioning in extreme weather or environments. To enhance the system's applicability to larger railway networks, efficiency, and scalability can be improved by offloading only the accelerated computation tasks to the FPGA while delegating subsequent complex operations to cloud servers, such as fault type recognition.

CONCLUSION

This work proposes a real-time railway fault detection architecture based on an edge computing platform, which integrates image

acquisition, model acceleration, and communication modules. To address the issue of imbalanced resource utilization that limits model implementation in resource-constrained FPGAs, we propose a DSP packing algorithm specifically optimized for BN layers. This method reduces the utilization of LUT resources by 10% without compromising detection accuracy, while DSP usage only increased by 4 and 7 slices, respectively. We built the RBPnet2 and RBPnet4 models using a binarized ResNet architecture, achieving detection accuracies of 90.9% and 93.4%, respectively. Moreover, the entire network structure was successfully deployed and optimized on embedded FPGA platforms. Experimental results demonstrate that the cost-energy efficiency of the RBPnet2 and RBPnet4 models outperforms high-performance devices like the 14700KF and RTX 4080 by factors of 101.9× and 33.8×, respectively. In embedded FPGAs, this system demonstrates real-time image classification capabilities while offering an excellent balance of cost-effectiveness and industrial applicability, which are crucial for consumer electronics.

■ REFERENCES

1. H. Sedjelmaci and N. Ansari, "On cooperative federated defense to secure multiaccess edge computing," *IEEE Consum. Electron. Mag.*, vol. 13, no. 4, pp. 24–31, Jul. 2024.
2. I. Ahmed, G. Jeon, and A. Ahmad, "Deep learning-based intrusion detection system for Internet of vehicles," *IEEE Consum. Electron. Mag.*, vol. 12, no. 1, pp. 117–123, Jan. 2023.
3. M. Xue, H. Wu, and R. Li, "Dnn migration in IoTs: Emerging technologies, current challenges, and open research directions," *IEEE Consum. Electron. Mag.*, vol. 12, no. 3, pp. 28–38, May 2023.
4. L. A. Maciel, M. A. Souza, and H. C. Freitas, "Energy-efficient CPU FPGA-based CNN architecture for intrusion detection systems," *IEEE Consum. Electron. Mag.*, vol. 13, no. 4, pp. 65–72, Jul. 2024.
5. M. Krishna S and T. Perumal, "Making buildings smarter and energy-efficient—using the Internet of Things platform," *IEEE Consum. Electron. Mag.*, vol. 10, no. 3, pp. 34–41, May 2021.
6. T. Xiao, T. Xu, and G. Wang, "Real-time detection of track fasteners based on object detection and fpga," *Microprocessors Microsyst.*, vol. 100, 2023, Art. no. 104863.
7. J. Li, Y. Fu, D. Yan, S. L. Ma, and C.-W. Sham, "An edge AI system based on FPGA platform for railway fault detection," in *Proc. IEEE 13th Glob. Conf. Consum. Electron.*, 2024, pp. 1387–1389.
8. Q. Xu, Q. Zhao, G. Yu, L. Wang, and T. Shen, "Rail defect detection method based on recurrent neural network," in *Proc. 39th Chin. Control Conf.*, 2020, pp. 6486–6490.
9. Y. Jiang, H. Wang, S. Chen, and G. Tian, "Visual quantitative detection of rail surface crack based on laser ultrasonic technology," *Optik*, vol. 237, 2021, Art. no. 166732.
10. J. Chen, Z. Liu, H. Wang, A. Núñez, and Z. Han, "Automatic defect detection of fasteners on the catenary support device using deep convolutional neural network," *IEEE Trans. Instrum. Meas.*, vol. 67, no. 2, pp. 257–269, Feb. 2018.
11. J. Sresakoolchai and S. Kaewunruen, "Railway defect detection based on track geometry using supervised and unsupervised machine learning," *Struct. Health Monit.*, vol. 21, no. 4, pp. 1757–1767, 2022, doi: [10.1177/14759217211044492](https://doi.org/10.1177/14759217211044492).
12. Y. Wu, Y. Qin, Y. Qian, F. Guo, Z. Wang, and L. Jia, "Hybrid deep learning architecture for rail surface segmentation and surface defect detection," *Comput.-Aided Civil Infrastructure Eng.*, vol. 37, no. 2, pp. 227–244, 2022.
13. Y. Zhang, J. Pan, X. Liu, H. Chen, D. Chen, and Z. Zhang, "Fracbnn: Accurate and FPGA-efficient binary neural networks with fractional activations," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays*, 2021, pp. 171–182, doi: [10.1145/3431920.3439296](https://doi.org/10.1145/3431920.3439296).
14. G. Yang, J. Lei, Z. Fang, Y. Li, J. Zhang, and W. Xie, "Hybnn: Quantifying and optimizing hardware efficiency of binary neural networks," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 17, no. 2, Apr. 2024, doi: [10.1145/3631610](https://doi.org/10.1145/3631610).
15. M. Ghasemzadeh, M. Samragh, and F. Koushanfar, "Rebnet: Residual binarized neural network," in *Proc. IEEE 26th Annu. Int. Symp. Field-Program. Custom Comput. Mach.*, 2018, pp. 57–64.
16. S. I. Eunus et al., "Ecarlnet: An efficient lstm-based ensembled deep neural network architecture for railway fault detection," *AI*, vol. 5, no. 2, pp. 482–503, 2024.

17. S. Liang, S. Yin, L. Liu, W. Luk, and S. Wei, "FP-BNN: Binarized neural network on FPGA," *Neurocomputing*, vol. 275, pp. 1072–1086, 2018.
18. P. Paz and M. Garrido, "Efficient implementation of complex multipliers on fpgas using dsp slices," *J. Signal Process. Syst.*, vol. 95, no. 4, pp. 543–550, Apr. 2023, doi: [10.1007/s11265-023-01867-7](https://doi.org/10.1007/s11265-023-01867-7).
19. J. Sommer, M. A. Özkan, O. Keszocze, and J. Teich, "Dsp-packing: Squeezing low-precision arithmetic into FPGA DSP blocks," in *Proc. 32nd Int. Conf. Field-Program. Log. Appl.*, 2022, pp. 160–166.
20. C. Hur and S. Kang, "Entropy-based pruning method for convolutional neural networks," *J. Supercomput.*, vol. 75, no. 6, pp. 2950–2963, Jun. 2019, doi: [10.1007/s11227-018-2684-z](https://doi.org/10.1007/s11227-018-2684-z).
21. R. Horaud and F. Dornaika, "Hand-eye calibration," *Int. J. Robot. Res.*, vol. 14, no. 3, pp. 195–210, 1995, doi: [10.1177/027836499501400301](https://doi.org/10.1177/027836499501400301).
22. S. Wang, G. Ananthanarayanan, Y. Zeng, N. Goel, A. Pathania, and T. Mitra, "High-throughput CNN inference on embedded arm big.little multicore processors," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 39, no. 10, pp. 2254–2267, Oct. 2020.
23. M. Chen, Z. Zhang, and H. Ren, "Design and verification of high performance memory interface based on axi bus," in *Proc. IEEE 21st Int. Conf. Commun. Technol.*, 2021, pp. 695–699.

Yulin Fu received the bachelor's degree in engineering from Northeast Forestry University, Harbin, China, and is currently working toward the Master of Science degree with the University of Auckland, Auckland, New Zealand. His research interests encompass hardware acceleration, edge computing, and deep learning. Contact him at yxue732@aucklanduni.ac.nz.

Dongwei Yan received the bachelor's degree in engineering from Northeast Forestry University,

Harbin, China, and is with the University of Auckland, Auckland. His research interest includes deep learning, edge computing, and hardware acceleration. Contact him at dyan232@aucklanduni.ac.nz.

Jiale Li received the bachelor's and master's degrees from the Civil Aviation University of China, Tianjin, China, and Nankai University, Tianjin, China, respectively. He is currently working toward the Ph.D. degree with the University of Auckland, Auckland, New Zealand. His research interests include computer architectures, processing-in-memory, and hardware acceleration. He is the corresponding author of this article. Contact him at jlj990@aucklanduni.ac.nz.

Sean Longyu Ma is a lecturer with the School of Computer Science. He received the bachelor's degree in communication from Harbin Engineering University, Harbin, China, in 2010, the master's degree in integrated circuit from Shanghai Jiao Tong University, Shanghai, China, in 2016, and the Ph.D. degree from the University of Auckland, Auckland, New Zealand, in 2023. His research areas are digital design using FPGA platforms, RISC-V core customization. Contact him at sean.ma@auckland.ac.nz.

Chiu-Wing Sham received the bachelor's degree in computer engineering and the MPhil. degree from The Chinese University of Hong Kong, Hong Kong, in 2000 and 2002, respectively, and the Ph.D. degree from the same university in 2006. He is a senior member of IEEE and now works at The University of Auckland as a Senior Lecturer. Contact him at b.sham@auckland.ac.nz.

Hong-fu Chou received the M.S. and Ph.D. degrees in communication engineering from National Taiwan University, Taipei, Taiwan, in 2006 and 2013, respectively. His research interests include digital circuit design, VLSI, FPGA implementation, quantum cryptography, and physical layer security. Contact him at hungpu.chou@uni.lu.

<https://ctsoc.ieee.org/>



CTSOC TECHNICAL TALKS

Building on the success of the series of talks organized in individual Technical Committees (TCs) over the last years, a new series of webinars has been launched - the *Technical Talks of CTSoc*. Every month, two of the 15 TCs will organize a 1-hour webinar, with the participation of two speakers representing their membership. Schedules is as follows (dates will be finalized month by month). Webinars are recorded and are published with the consent of speakers and participating audience.

